CPSC 250
Testing Code with Unit Tests and Timers

Edward Brash

1 Motivation

As programs become more complex, it becomes harder to know whether they are correct.
It is also harder to know whether one algorithm is better than another.
Two useful tools help with these questions:

1. unit tests,
2. timing code.

Unit tests help answer:

> Does my program do what I think it does?

Timers help answer:

> How long does this algorithm take to run?

Together, testing and timing are essential parts of serious software development.

2 Why Unit Tests Matter

Unit testing is not just something used by auto-graders.
Unit tests are widely used in professional software development.
A major use case is **continuous integration**, often abbreviated as CI.

3 Continuous Integration

Imagine a large project hosted on GitHub.
Many people may contribute to the project.
Each person may change one part of the code.
The danger is that a change in one file may accidentally break another part of the project.
Continuous integration systems help by automatically running tests whenever code is updated.

> The purpose is to detect broken behavior quickly.

4 Object-Oriented Programming and Testing

Unit tests are especially important in object-oriented programming.

If a class is used by many other parts of a program, then changing one method in that class can accidentally break code elsewhere.

A good set of unit tests helps make sure that the class still behaves correctly after changes.

5 A Simple Example: Circle Class

Suppose we define a simple `Circle` class.

```
class Circle:

    def __init__(self, radius):
        self.radius = radius

    def compute_area(self):
        return 3.14159265 * self.radius ** 2

    def compute_circumference(self):
        return 2.0 * 3.14159265 * self.radius
```

This class has:

- a constructor,
- a method to compute area,
- a method to compute circumference.

6 The `unittest` Module

Python includes a built-in testing module called:

```
unittest
```

We import it using:

```
import unittest
```

7 Creating a Test Class

A test class should inherit from:

```
unittest.TestCase
```

Example:

```
class TestCircle(unittest.TestCase):
    ...
```

This inheritance gives the test class access to useful testing methods such as:

- `assertEqual()`,

- `assertLess()`,
- `assertGreater()`,
- `assertAlmostEqual()`.

8 Writing a Test Method

Test methods are functions inside the test class.

Important rule:

> Test method names must begin with `test_`.

For example:

```
def test_area(self):
    ...
```

The `unittest` framework automatically finds and runs methods whose names begin with `test_`.

9 Testing the Area

A first test checks that a circle of radius zero has area zero.

```
def test_area(self):

    c = Circle(0)

    self.assertEqual(c.compute_area(), 0.0)
```

This test should pass.

10 Testing a Nonzero Area

For a circle of radius 5:

$$A = \pi r^2$$

so:

$$A = \pi(5)^2 = 25\pi \approx 78.5398$$

We can test this:

```
def test_area(self):

    c = Circle(0)
    self.assertEqual(c.compute_area(), 0.0)

    c = Circle(5)
    self.assertAlmostEqual(c.compute_area(), 78.5398, places=4)
```

For floating point values, `assertAlmostEqual()` is often better than `assertEqual()`.

11 Testing for Bad Behavior

Sometimes tests are designed to check that something is *not* true.

For example, the area of a circle with positive radius should not be less than zero.

```
def test_stupid(self):

    c = Circle(5)

    self.assertLess(c.compute_area(), 0.0)
```

This test is expected to fail.

That is useful: it shows that the test framework is actually catching incorrect statements.

12 Running the Tests

At the bottom of the test file, we write:

```
if __name__ == "__main__":
    unittest.main()
```

This runs the tests when the file is executed directly.

13 Complete Unit Test Example

```
import unittest


class Circle:

    def __init__(self, radius):
        self.radius = radius

    def compute_area(self):
        return 3.14159265 * self.radius ** 2

    def compute_circumference(self):
        return 2.0 * 3.14159265 * self.radius


class TestCircle(unittest.TestCase):

    def test_area(self):

        c = Circle(0)
        self.assertEqual(c.compute_area(), 0.0)

        c = Circle(5)
        self.assertAlmostEqual(c.compute_area(), 78.5398, places=4)

    def test_stupid(self):

        c = Circle(5)
        self.assertLess(c.compute_area(), 0.0)
```

```
if __name__ == "__main__":
    unittest.main()
```

14 Common Assert Methods

Method	Checks That
`assertEqual(a, b)`	$a == b$
`assertNotEqual(a, b)`	$a \neq b$
`assertTrue(x)`	`bool(x)` is `True`
`assertFalse(x)`	`bool(x)` is `False`
`assertIs(a, b)`	a is b
`assertIsNot(a, b)`	a is not b
`assertIsNone(x)`	x is `None`
`assertIsNotNone(x)`	x is not `None`
`assertIn(a, b)`	a in b
`assertNotIn(a, b)`	a not in b
`assertAlmostEqual(a, b)`	rounded difference is zero
`assertGreater(a, b)`	$a > b$
`assertGreaterEqual(a, b)`	$a \geq b$
`assertLess(a, b)`	$a < b$
`assertLessEqual(a, b)`	$a \leq b$

15 Timing Code

Once we have multiple possible algorithms, correctness is not the only question.

We also care about performance.

For example, if two algorithms both produce the correct answer, we may want to know which one is faster.

16 The `time` Module

Python includes a built-in module called:

```
time
```

This module can be used directly, but it can feel cumbersome.

A useful approach is to create a small wrapper class around the timing functionality.

17 The Timer Wrapper

Suppose we have a file called:

```
timer.py
```

containing a `Timer` class.

Then we can import it using:

```
from timer import Timer
```

18 Using the Timer

The basic pattern is:

```
time1 = Timer()

time1.start()

# do things here

elapsed_time1 = time1.stop()
```

The timer measures how much time passes between:

```
start()
```

and:

```
stop()
```

19 Example: Timing a Loop

```
from timer import Timer

timer = Timer()

timer.start()

total = 0

for i in range(1000000):
    total += i

elapsed_time = timer.stop()

print(f"Elapsed time: {elapsed_time}")
```

20 Why Timing Matters

Timing code helps us compare algorithms in realistic situations.

For example, suppose we have:

- a recursive Fibonacci function,
- an iterative Fibonacci function,
- a formula-based Fibonacci function.

All three may produce the same answer for small n, but their runtime behavior can be dramatically different.

21 Testing vs Timing

Testing and timing answer different questions.

Tool	Main Question
Unit tests	Is the code correct?
Timers	How long does the code take?

Both are important.
Correct but very slow code may not be useful.
Fast but incorrect code is also not useful.

22 Good Workflow

A good development workflow is:

1. Write the code.
2. Write unit tests.
3. Run the tests.
4. Fix errors until the tests pass.
5. Time the code.
6. Compare alternative algorithms.
7. Choose an implementation based on correctness, clarity, and performance.

23 Key Takeaways

- Unit tests check whether code behaves correctly.
- Unit tests are important beyond auto-grading.
- Continuous integration systems use tests to detect broken code automatically.
- Test classes inherit from `unittest.TestCase`.
- Test methods must begin with `test_`.
- `unittest.main()` runs the tests.
- Timers measure algorithm performance.
- Correctness and performance are different but complementary concerns.